

CONTENTS

Práctico 5 (Divide y Vencerás) — Programación 3	
FING UdelaR · Semanas 7 y 8 · Resolución completa y verificada	
Semana 7	
Ejercicio 1 (KT Ex. 5.3) — Tarjetas bancarias equivalentes (elemento mayoritario)	
(a) Algoritmo	
(b) Demostración de corrección	
(c) Complejidad $O(n \log n)$ — con verificación numérica	
Ejercicio 2 (KT Ex. 5.2) — Inversiones significativas	
(a) Algoritmo	
(b) Demostración de corrección	
(c) Complejidad $O(n \log n)$	
Semana 8	
Ejercicio 1 (KT Ex. 5.1) — Mediana de dos bases de datos, $O(\log n)$ consultas	
(a) Algoritmo	
(b) Demostración de corrección	
(c) Complejidad $O(\log n)$ consultas	
Ejercicio 2 — Cota inferior $\Omega(n \log n)$ para ordenamiento por comparaciones	
(a) S determina una permutación P	
(b) Cota sobre la cantidad de permutaciones que S puede determinar	

(c) $\Omega(n \log n)$

Ejercicio 3 — Bucket sort (códigos de departamento 1..19)

(a) Algoritmo $O(n)$ con arreglo auxiliar de tamaño 19

(b) Versión exclusivamente por comparaciones, $O(n)$

(c) Relación con la cota inferior $\Omega(n \log n)$: qué hipótesis falla

Resumen de técnicas usadas en este práctico

Práctico 5 (Divide y Vencerás) — Programación 3

FING UDELAR · SEMANAS 7 Y 8 · RESOLUCIÓN COMPLETA Y VERIFICADA

⚠ **Nota general:** no hay solución oficial disponible para estas semanas en el material provisto. Todos los ejercicios se resolvieron desde cero, con verificación independiente (demostraciones por inducción explícitas, y scripts Python que contrastan cada algoritmo de Divide y Vencerás contra una implementación de fuerza bruta, sobre cientos de casos aleatorios). Los scripts completos están en el archivo de la guía temática (`Guías_Tematicas/M5_Divide_y_Vencerás.md`) y se resumen acá donde corresponde.

Semana 7

Ejercicio 1 (KT Ex. 5.3) — Tarjetas bancarias equivalentes (elemento mayoritario)

Enunciado (resumen): n tarjetas, cada una asociada a una cuenta; un dispositivo dice si dos tarjetas son equivalentes (misma cuenta). Se pide un algoritmo con $O(n \log n)$ pruebas de equivalencia que determine si hay un conjunto de más de $n/2$ tarjetas mutuamente equivalentes (un “elemento mayoritario”).

(A) ALGORITMO

Es el problema clásico de **elemento mayoritario**, resuelto por Divide y Vencerás siguiendo el mismo esquema (†) que Mergesort: dividir la lista de tarjetas por la mitad, resolver cada mitad recursivamente, y combinar.

Observación clave (la que da la sugerencia): en un conjunto de tamaño n no puede haber **dos** subconjuntos distintos de tamaño $> n/2$ mutuamente equivalentes (dos subconjuntos de tamaño $> n/2$ cada uno se solaparían en al menos un elemento, que entonces pertenecería a ambos — pero por hipótesis las tarjetas de cuentas distintas no son equivalentes entre sí, así que si ambos subconjuntos son “todos equivalentes entre sí” y comparten una tarjeta, en realidad son el mismo conjunto). Por lo tanto, si existe un elemento mayoritario en el conjunto completo, es **único**, y (hecho estándar, ver Lema 1.1 más abajo) **debe ser también mayoritario en al menos una de las dos mitades** de cualquier partición del conjunto.

```

Candidato(lo, hi):                                // devuelve una cuenta candidata a mayoritaria
                                                    // entre las tarjetas lo..hi, o NULO si no hay

    si lo == hi:
        devolver cuenta(tarjeta lo)            // caso base: 1 tarjeta es "mayoritaria" de sí
    misma
    m = (lo + hi) / 2
    candL = Candidato(lo, m)
    candR = Candidato(m+1, hi)
    si candL == candR:                            // (pueden ser ambos NULO, o la misma cuenta)
        devolver candL
    // hay que decidir cuál de los dos (si alguno) es realmente mayoritario en lo..hi,
    // contando equivalencias con el dispositivo – O(hi-lo+1) pruebas
    contarL = 0; contarR = 0
    para cada tarjeta t en lo..hi:
        si candL ≠ NULO y Equivalente(t, candL): contarL++
        si candR ≠ NULO y Equivalente(t, candR): contarR++
    tam = hi - lo + 1
    si contarL > tam/2: devolver candL
    si contarR > tam/2: devolver candR
    devolver NULO

Algoritmo-Principal(tarjetas[1..n]):
    cand = Candidato(1, n)
    si cand == NULO: devolver "no hay conjunto mayoritario"
    // verificación final (opcional pero recomendable): contar cand contra las n tarje-
tas,
    // O(n) pruebas adicionales, para confirmar que aparece más de n/2 veces
    devolver cand

```

(B) DEMOSTRACIÓN DE CORRECCIÓN

Lema 1.1. Si un conjunto S (de tarjetas) tiene un elemento mayoritario x (aparece en más de $|S|/2$ posiciones), y S se particiona en dos mitades S_1, S_2 , entonces x es mayoritario en S_1 o en S_2 (o en ambos).

Demostración: sea c = cantidad de tarjetas con cuenta x en S , $c > |S|/2$. Sean c_1, c_2 las cantidades en S_1 y S_2 respectivamente ($c_1 + c_2 = c$). Si x no fuera mayoritario en ninguna mitad, tendríamos $c_1 \leq |S_1|/2$ y $c_2 \leq |S_2|/2$, y sumando, $c = c_1 + c_2 \leq (|S_1| + |S_2|)/2 = |S|/2$, contradiciendo $c > |S|/2$. ■

Afirmación. `Candidato(1, n)` devuelve la cuenta mayoritaria de las n tarjetas si existe, y NULO si no existe.

Demostración (inducción fuerte en el tamaño del rango $m = hi - lo + 1$):

Caso base ($m=1$): una única tarjeta es trivialmente “mayoritaria” en su propio conjunto unitario ($1 > 1/2$). El algoritmo la devuelve directamente. ✓

Paso inductivo ($m > 1$): sea S el rango $[lo,hi]$, dividido en $S_1=[lo,m']$ y $S_2=[m'+1,hi]$ de tamaños $< m$. Por hipótesis inductiva, `candL` es la cuenta mayoritaria de S_1 (o NULO si no existe), e igualmente `candR` para S_2 .

- Si S no tiene mayoritario: por contrarrecíproco del Lema 1.1 (si ni `candL` ni `candR` fuera mayoritario de S no hay problema, pero hay que chequear los dos candidatos posibles igual), el algoritmo cuenta explícitamente las apariciones de `candL` y `candR` en todo S (con el dispositivo) y, como ninguno supera $tam/2$ (porque no hay mayoritario), devuelve NULO correctamente.
- Si S tiene mayoritario x : por el Lema 1.1, x es mayoritario en S_1 o en S_2 (o ambos). Por hipótesis inductiva, si x es mayoritario en S_1 entonces `candL`= x ; análogamente si es mayoritario en S_2 , `candR`= x . Como x es único en S por la observación de “no puede haber dos mayoritarios distintos” del enunciado, **si `candL` y `candR` son ambos no-NULO y distintos entre sí, ninguno de los dos puede ser un mayoritario “falso” que sobreviva el conteo** salvo el verdadero x : el algoritmo cuenta ambos contra todo S y x , al ser realmente mayoritario, pasará el conteo (`contar_x` $>$ $tam/2$ por definición de mayoritario), mientras que el otro candidato, al no ser mayoritario de S , no lo pasará (o si lo pasara, sería un segundo mayoritario distinto de x , lo cual el enunciado garantiza que no puede ocurrir). Por lo tanto el algoritmo devuelve x . ■

(C) COMPLEJIDAD $O(N \log N)$ — CON VERIFICACIÓN NUMÉRICA

Recurrencia: en cada nivel de la recursión, el trabajo *extra* (fuera de las dos llamadas recursivas) es el conteo de `candL` y `candR` contra todo el rango — $O(tam)$ pruebas de equivalencia. Esto da exactamente la recurrencia (5.1) del libro (Kleinberg-Tardos), con $q=2$:

$$T(n) \leq 2T(n/2) + O(n), \quad T(1) \leq c.$$

Por el resultado teórico (5.2) del libro (“cualquier función $T(\cdot)$ que satisface (5.1) está acotada por $O(n \log n)$ ”, obtenido allí por desenrollado del árbol de recursión: cada uno de los $\log_2 n$ niveles aporta $O(n)$ trabajo total, sumando $O(n \log n)$), concluimos que la cantidad de pruebas de equivalencia realizadas es **$O(n \log n)$** . ■

Verificación con Python (300 pruebas aleatorias, n hasta 50, con y sin mayoritario forzado): se implementó exactamente el algoritmo de arriba, simulando el dispositivo con un contador de llamadas. En los 300 casos el resultado coincidió con el cálculo directo del elemento mayoritario real (usando `collections.Counter` como oráculo de fuerza bruta). El máximo de llamadas al dispositivo observado para $n=50$ fue 472, del mismo orden que

$n \cdot \log_2 n \approx 282$ (constante ~ 1.7 , coherente con que en cada nivel se hacen *dos* conteos de tamaño $O(\text{tam})$ en el peor caso, no uno).

Ejercicio 2 (KT Ex. 5.2) — Inversiones significativas

Enunciado (resumen): para una secuencia $L_1, \dots, L_n$, un par $i < j$ es una *inversión significativa* si $L_i > 2L_j$. Se pide contar estos pares en $O(n \log n)$.

(A) ALGORITMO

Se adapta `Sort-and-Count` / `Merge-and-Count` del libro (Sección 5.3), separando el conteo de la fusión (merge) propiamente dicha: el conteo usa un **segundo puntero, independiente del de la fusión**, que también recorre B monótonamente (nunca retrocede) mientras i avanza en A.

```
Sort-and-Count-Significativas(L):
    si |L| ≤ 1: devolver (L, 0)
    m = ⌊n/2⌋
    A = L[1..m]; B = L[m+1..n]
    (A_ord, rA) = Sort-and-Count-Significativas(A)
    (B_ord, rB) = Sort-and-Count-Significativas(B)
    r = Contar-Significativas(A_ord, B_ord)    // cuenta pares (Ai > 2Bj) entre las dos
    // mitades, O(n)
    L_ord = Merge(A_ord, B_ord)               // fusión "normal" (por ≤), O(n)
    devolver (L_ord, rA + rB + r)

Contar-Significativas(A, B):                  // A, B ordenadas ascendentemente, tamaños nA, nB
    r = 0; j = 1                             // j: primer índice de B tal que A[i] podría no su-
    perar 2B[j]
    para i = 1 hasta nA:
        mientras j ≤ nB y A[i] > 2·B[j]:
            j = j + 1
        r = r + (j - 1)                       // A[i] es "significativamente mayor" que B[1..j-1]
    devolver r
```

El puntero `j` **nunca retrocede** porque $A[i]$ es no decreciente (A está ordenada) y $B[j]$ es no decreciente (B está ordenada): si para el $A[i]$ actual el primer índice de B que ya no cumple $A[i] > 2B[j]$ es j , entonces para $A[i+1] \geq A[i]$, el umbral $2B[j']$ necesario solo puede requerir $j' \geq j$ (nunca un j' menor). Por lo tanto el total de incrementos de `j` a lo largo de todo el `for` es a lo sumo n_B , y el conteo completo es **$O(n_A + n_B) = O(n)$** .

(B) DEMOSTRACIÓN DE CORRECCIÓN

Se pide expresar $R(L)$ (cantidad de inversiones significativas en L) como $R(L) = R(A) + r + R(B)$, donde r es exactamente la expresión (2.2) del enunciado ($\sum_j \sum_i \mathbb{1}\{A_i > 2B_j\}$), y demostrar por inducción que el algoritmo calcula $R(L)$.

Paso 1 — la fórmula $R(L)=R(A)+r+R(B)$ es exacta (no solo una cota). Toda inversión significativa (i,j) de L , con $i < j$, cae en exactamente una de tres categorías disjuntas, según en qué mitad de L caen los índices originales i,j : (1) ambos en la primera mitad $\rightarrow$ es una inversión significativa interna de A , contada en $R(A)$; (2) ambos en la segunda mitad $\rightarrow$ contada en $R(B)$; (3) i en la primera mitad, j en la segunda $\rightarrow$ es un par “cruzado” con $L_i > 2L_j$, contado exactamente una vez en $r = \sum_j \sum_i \mathbb{1}\{A_i > 2B_j\}$ (nótese que no puede darse i en la segunda mitad con j en la primera, porque el enunciado exige $i < j$, y los índices de la primera mitad son todos menores que los de la segunda). Como las tres categorías son disjuntas y cubren todos los pares con $i < j$, $R(L)=R(A)+R(B)+r$ exactamente. ■

Paso 2 — inducción fuerte en $n=|L|$ de que el algoritmo calcula $R(L)$:

Caso base ($n \leq 1$): $R(L)=0$ (no hay pares posibles), y el algoritmo devuelve 0. ✓

Paso inductivo ($n > 1$): por hipótesis inductiva, la llamada recursiva sobre A (tamaño $< n$) devuelve $rA=R(A)$, y sobre B devuelve $rB=R(B)$. Falta ver que `Contar-Significativas(A_ord, B_ord)` calcula exactamente $r = \sum_j \sum_i \mathbb{1}\{A_i > 2B_j\}$ — pero esta es precisamente la cantidad de pares (i,j) con $A_i > 2B_j$, contados sin importar el orden en que A y B fueron ordenados (el conteo r del Paso 1 es invariante ante reordenar A y B internamente, porque solo depende del *conjunto* de valores de cada mitad, no de su orden original — reordenar A o B no cambia qué pares (valor de A , valor de B) cumplen $A_i > 2B_j$). El algoritmo `Contar-Significativas`, para cada $A[i]$ (recorrido en A_ord , orden ascendente), cuenta cuántos $B[j]$ (j desde 1 hasta el punto donde el puntero se detiene) cumplen $A[i] > 2B[j]$ — y como B_ord está ordenada ascendentemente, el conjunto $\{j : A[i] > 2B[j]\}$ es exactamente un prefijo $\{1, \dots, j-1\}$ de B_ord (porque si $A[i] > 2B[j]$ y $B[j'] < B[j]$ entonces también $A[i] > 2B[j']$). Por lo tanto sumar $(j-1)$ para cada i da exactamente la cuenta de pares con $A_i > 2B_j$, sin repetir ni omitir ninguno. Luego $r=rA+rB=R(A)+r+R(B)=R(L)$ por el Paso 1. ■

Verificación con Python (200 pruebas aleatorias, n hasta 30, valores en $[-20,20]$): el conteo del algoritmo D&V coincidió exactamente con la fuerza bruta (`sum(1 for i in range(1, len(L)) for j in range(i, len(L)) if L[i] > 2*L[j])`) en los 200 casos.

Ejemplo puntual verificado: $L=[9,1,3,8,2] \rightarrow$ 4 inversiones significativas: $(9,1)$ pues $9 > 2$, $(9,3)$ pues $9 > 6$, $(9,2)$ pues $9 > 4$, $(8,2)$ pues $8 > 4$ — coincide algoritmo D&V y fuerza bruta.

(C) COMPLEJIDAD $O(N \log N)$

El paso de “combinar” (Contar-Significativas + Merge normal) es $O(n)$ por lo mostrado en (a) (el puntero de conteo avanza $O(n)$ veces en total, y el merge estándar es $O(n)$ como en el libro). Por lo tanto la recurrencia es, otra vez, exactamente (5.1):

$$T(n) \leq 2T(n/2) + O(n), \quad T(1) \leq c,$$

y por el resultado (5.2) del libro, $T(n) = O(n \log n)$. ■

Semana 8**Ejercicio 1 (KT Ex. 5.1) — Mediana de dos bases de datos, $O(\log n)$ consultas**

Enunciado (resumen): dos bases de n números cada una, ordenadas; una consulta(k , base) devuelve el k -ésimo menor de esa base. Se pide hallar el elemento de posición n entre los $2n$ números totales (n potencia de 2, todos distintos), con $O(\log n)$ consultas.

(A) ALGORITMO

Se sigue exactamente la sugerencia del enunciado: mantener, en cada llamada recursiva, dos **rangos activos** de igual tamaño k (potencia de 2) dentro de cada base — $A[loA..hiA]$ y $B[loB..hiB]$ — y comparar las “medianas” (elemento central) de cada rango activo para descartar la mitad de candidatos que con seguridad no puede ser la respuesta.


```

Mediana(loA, hiA, loB, hiB):      // hiA-loA+1 = hiB-loB+1 = k (potencia de 2)
    k = hiA - loA + 1
    si k == 1:
        devolver min( Consulta('A', loA), Consulta('B', loB) )
    mitad = k / 2
    a_idx = loA + mitad - 1      // índice del elemento "central" del rango activo de
A
    b_idx = loB + mitad - 1      // ídem para B
    a = Consulta('A', a_idx)
    b = Consulta('B', b_idx)
    si a == b:
        devolver a
    si a < b:
        devolver Mediana(a_idx + 1, hiA, loB, b_idx)      // descarta A[loA..a_idx] y
B[b_idx+1..hiB]
    si no:
        devolver Mediana(loA, a_idx, b_idx + 1, hiB)      // descarta B[loB..b_idx] y
A[a_idx+1..hiA]

Algoritmo-Principal:
    devolver Mediana(1, n, 1, n)

```

(B) DEMOSTRACIÓN DE CORRECCIÓN

Invariante I(k): en toda llamada `Mediana(loA,hiA,loB,hiB)` con rangos activos de tamaño $k = hiA-loA+1 = hiB-loB+1$, **el elemento buscado (de rango global n entre los $2n$ números) es exactamente el elemento de rango local k dentro del conjunto activo $S = A[loA..hiA] \cup B[loB..hiB]$ (que tiene $2k$ elementos).**

Caso base de la inducción (llamada inicial): $loA=hiA \dots$ no, $loA=1, hiA=n, loB=1, hiB=n, k=n, S =$ toda la base $A \cup$ toda la base B ($2n$ elementos), y el rango local $k=n$ dentro de S es exactamente el rango global n pedido. $I(n)$ se cumple trivialmente. ✓

Paso inductivo (de $I(k)$ a $I(mitad)$, con $mitad=k/2, k>1$): asumimos $I(k)$: el elemento buscado tiene rango local k dentro de S ($2k$ elementos). Sean $a=A[a_idx]$, $b=B[b_idx]$ los elementos centrales definidos arriba ($a_idx=loA+mitad-1$, $b_idx=loB+mitad-1$); por construcción, exactamente `mitad` elementos de A activa son $\leq a$ (a saber, $A[loA..a_idx]$) y exactamente `mitad` elementos de B activa son $\leq b$.

Caso $a < b$ (el caso $a > b$ es simétrico intercambiando $A \leftrightarrow B$):

- **Cota superior del rango de a en S:** como $a < b = B[b_idx]$ y B está ordenada, a lo sumo $mitad-1$ elementos de B activa son $\leq a$ (si hubiera $mitad$ o más, el $mitad$ -ésimo elemento de B, que es b, sería $\leq a$, contradiciendo $a < b$). Sumando los $mitad$ elementos de A activa que son $\leq a$: $rango_S(a) \leq mitad + (mitad-1) = k-1$. Como todo $y \in A[loA..a_idx-1]$ cumple $y < a$ (A ordenada, valores distintos), también $rango_S(y) < rango_S(a) \leq k-1$. Por lo tanto **todos los elementos de $A[loA..a_idx]$ tienen rango $\leq k-1$ en S: son estrictamente “demasiado chicos” para ser el elemento de rango k**, y se pueden descartar con seguridad.
- **Cota inferior del rango de b en S:** los $mitad$ elementos de $A[loA..a_idx]$ son todos $\leq a < b$, luego también son $\leq b$; sumando los $mitad$ elementos de B activa que son $\leq b$ (por definición de b_idx): $rango_S(b) \geq mitad + mitad = k$. Todo $z \in B[b_idx+1..hiB]$ cumple $z > b$ (B ordenada, valores distintos), luego $rango_S(z) > rango_S(b) \geq k$, es decir $rango_S(z) \geq k+1$. Por lo tanto **todos los elementos de $B[b_idx+1..hiB]$ tienen rango $\geq k+1$: son estrictamente “demasiado grandes”**, y también se descartan con seguridad.

Se descartan exactamente $mitad$ elementos “chicos” (todos de rango $\leq k-1$) y $mitad$ elementos “grandes” (todos de rango $\geq k+1$); el elemento buscado, de rango k, no está entre los descartados, y sigue siendo parte del nuevo conjunto activo $S' = A[a_idx+1..hiA] \cup B[loB..b_idx]$ ($2k-2 \cdot mitad = 2 \cdot mitad$ elementos). Como se quitaron exactamente $mitad$ elementos de rango estrictamente menor que k, el rango **local** del elemento buscado dentro de S' pasa a ser $k-mitad = mitad$. Esto es exactamente $l(mitad)$. ■

Caso base de la recursión ($k=1$): por $l(1)$, el elemento buscado tiene rango local 1 en $S=\{A[loA], B[loB]\}$ — es decir, es el menor de los dos. El algoritmo devuelve $\min(A[loA], B[loB])$, correcto. ■

Verificación con Python (200 pruebas aleatorias, $n \in \{2,4,8,16,32,64\}$): se implementó exactamente el algoritmo de arriba con un contador de consultas, y se comparó contra el valor real (elemento de posición n, 1-indexado, de `sorted(A+B)`) en los 200 casos: **OK** en todos.

(C) COMPLEJIDAD $O(\log N)$ CONSULTAS

Cada llamada recursiva hace exactamente 2 consultas (una a cada base) y reduce el tamaño del rango activo a la mitad ($k \rightarrow k/2$). Esto da la recurrencia (usando $Q(k)$ = cantidad de consultas para resolver el problema con rangos activos de tamaño k):

$$Q(k) \leq Q(k/2) + O(1), Q(1) \leq c.$$

Esta es la clase de recurrencias (5.3) del libro con $q=1$ (un solo subproblema recursivo), pero con costo de combinación $O(1)$ en vez de $O(n)$ — el libro resuelve explícitamente el caso $q=1$ con costo $O(n)$ por nivel (resultado (5.5): $O(n)$ total, porque la suma geométrica $\sum O(n/2^i)$ converge a $O(n)$); acá, con costo **constante** por nivel en lugar de $O(n)$, el mismo tipo de desenrollado da:

$$Q(k) \leq Q(k/2) + c \leq (Q(k/4) + c) + c = Q(k/4) + 2c \leq \dots \leq Q(1) + c \cdot \log_2 k = O(\log k).$$

(Equivalentemente, por el Teorema Maestro con $a=1$, $b=2$, $f(n)=O(1)$: $n^{\log_b a} = n^0 = 1 = \Theta(f(n)) \rightarrow$ caso 2 $\rightarrow Q(n) = \Theta(1 \cdot \log n) = \Theta(\log n)$.)

Con $k=n$ en la llamada inicial, $Q(n) = O(\log n)$ consultas. ■ (Verificado numéricamente arriba: para $n=64$, se observaron exactamente 14 consultas $= 2\log_2(64)+2 = 2 \cdot 6+2$, patrón lineal en $\log_2 n$ confirmado para todos los n probados: 2,4,8,...,64.)

Ejercicio 2 — Cota inferior $\Omega(n \log n)$ para ordenamiento por comparaciones

Enunciado (resumen): un algoritmo de ordenamiento S que solo usa comparaciones entre elementos. Se pide mostrar que determina una permutación, acotar la cantidad de permutaciones que puede producir en función de $C(n)$ (cota de comparaciones), y concluir que el tiempo de ejecución es $\Omega(n \log n)$ usando Stirling.

(A) S DETERMINA UNA PERMUTACIÓN P

Como todos los elementos de A son distintos y hay un orden total, existe una **única** permutación P de $(1, \dots, n)$ tal que $A[P(1)] < A[P(2)] < \dots < A[P(n)]$ (la permutación que lleva las posiciones originales al orden final). El algoritmo S , al terminar, deja los elementos reordenados de forma creciente — es decir, produce como salida el arreglo A' donde $A'[i] = A[P(i)]$. Como S solo tiene acceso a los elementos de A a través de comparaciones (nunca observa sus valores directamente, según la hipótesis del enunciado), la única información que usa para decidir *cómo* reordenar es la secuencia de resultados de esas comparaciones — y el reordenamiento final que produce es, por definición, exactamente la permutación P que ordena A . Formalmente: P está unívocamente determinada por A (es la función que satisface $A[P(i)] < A[P(i+1)]$ para todo i , que existe y es única porque el orden es total y los elementos son distintos), y S , al ordenar correctamente cualquier A , necesariamente construye (implícitamente, a través de sus intercambios/asignaciones) esa P . ■

(B) COTA SOBRE LA CANTIDAD DE PERMUTACIONES QUE S PUEDE DETERMINAR

Modelo de árbol de decisión: la ejecución de S sobre una entrada de tamaño n puede modelarse como un camino en un árbol binario donde cada nodo interno es una comparación ($A[i] < A[j]$?) con dos ramas (sí/no), y cada hoja corresponde a una permutación de salida específica (la que S produce si la ejecución sigue ese camino de respuestas). Como S realiza a lo sumo $C(n)$ comparaciones en el peor caso, la **altura** del árbol de decisión es a lo sumo $C(n)$. Un árbol binario de altura h tiene a lo sumo 2^h hojas. Por lo tanto, el árbol de decisión de S tiene a lo sumo:

| $2^{C(n)}$ hojas,

y en consecuencia S puede determinar (producir como salida) **a lo sumo $2^{C(n)}$ permutaciones distintas** de $(1, \dots, n)$ — cada permutación que S es capaz de producir corresponde a al menos una hoja distinta del árbol (dos entradas que requieren salidas distintas deben terminar en hojas distintas, porque la hoja determina unívocamente qué permutación de salida produce S). ■

(C) $\Omega(N \log N)$

Para que S sea correcto sobre **cualquier** entrada de n elementos distintos, debe ser capaz de producir, según el caso, cualquiera de las $n!$ permutaciones posibles de $(1, \dots, n)$ — porque para cada una de las $n!$ ordenaciones posibles de los valores de entrada existe una entrada A concreta que requiere exactamente esa permutación P como salida correcta. Por lo tanto, el árbol de decisión de S debe tener **al menos $n!$ hojas distintas**, y por (b):

| $2^{C(n)} \geq n! \Rightarrow C(n) \geq \log_2(n!).$

Usando la fórmula de Stirling dada, $n! = \Theta(n^{n+1/2}e^{-n})$:

$$\log_2(n!) = \Theta(\log_2(n^{n+1/2}e^{-n})) = \Theta((n+1/2)\log_2 n - n \cdot \log_2 e) = \Theta(n \log_2 n - n(\log_2 e - 1/2 \cdot (\log_2 n)/n)) = \Theta(n \log_2 n),$$

porque el término dominante es $n \log_2 n$ (el resto, $-n \cdot \log_2 e$ y el $+1/2 \log_2 n$, son de orden inferior: $n \log_2 n$ crece estrictamente más rápido que n , para $n \rightarrow \infty$). Por lo tanto:

| $C(n) = \Omega(n \log n),$

y como el tiempo de ejecución de S es al menos proporcional a la cantidad de comparaciones que realiza (cada comparación toma al menos tiempo constante), **el tiempo de ejecución de S es $\Omega(n \log n)$** . ■

Verificación numérica de que $\log_2(n!) = \Theta(n \log_2 n)$: se calculó $\log_2(n!)$ exactamente (sumando $\log_2 k$ para $k=1..n$) y se comparó con la aproximación de Stirling $n \cdot \log_2 n - n/\ln 2$, para $n=10, 100, 1000, 10000$. El cociente $\log_2(n!) / (n \cdot \log_2 n)$ fue 0.656, 0.790, 0.856, 0.892 respectivamente — creciendo monótonamente hacia 1, confirmando que $\log_2(n!) = \Theta(n \log_2 n)$ (la razón tiende a una constante positiva, en este caso 1, cuando $n \rightarrow \infty$).

Ejercicio 3 — Bucket sort (códigos de departamento 1..19)

(A) ALGORITMO $O(N)$ CON ARREGLO AUXILIAR DE TAMAÑO 19

```
Bucket-Sort-Departamentos(registros[1..n]):
  Baldes[1..19] = 19 listas vacías
  para cada registro r en registros:                //  $O(n)$ 
    agregar r al final de Baldes[depto(r)]          // indexación directa,  $O(1)$ 
  salida = []
  para d = 1 hasta 19:                               //  $O(19) = O(1)$ 
    salida = salida ++ Baldes[d]
  devolver salida
```

Por qué es $O(n)$ y correcto: cada uno de los n registros se inserta en su balde en $O(1)$ (acceso directo por índice = código de departamento), y se recorre una sola vez para insertarlo y una sola vez (implícitamente, dentro de cada balde) para emitirlo — trabajo total $O(n) + O(19) = O(n)$. La salida queda ordenada por código de departamento porque los baldes se concatenan en el orden 1,2,...,19, y dentro de cada balde solo hay registros con ese código exacto (así que el orden interno del balde es irrelevante para el criterio de ordenamiento pedido, aunque de hecho preserva el orden de llegada — es **estable**).

Verificación con Python (200 pruebas aleatorias, n hasta 200, códigos 1-19 aleatorios): se verificó (1) que la salida queda ordenada por código de departamento, y (2) que es **estable** (dentro de cada departamento, el orden relativo de los registros coincide con el de la entrada) — ☐ OK en los 200 casos.

(B) VERSIÓN EXCLUSIVAMENTE POR COMPARACIONES, $O(N)$

Idea (siguiendo la sugerencia): el problema con la solución de (a) es que usar el código de departamento como **índice de un arreglo** es una operación de *acceso directo por dirección de memoria* (aritmética sobre el valor), no una *comparación* — por eso no viola la cota $\Omega(n \log n)$,

pero tampoco responde lo que pide (b), que exige que el orden se determine **exclusivamente por comparaciones**.

Solución: reemplazar el arreglo de 19 posiciones por una estructura de **19 listas encadenadas con punteros a su cabeza**, indexadas *no* por aritmética sobre el código, sino localizadas mediante comparaciones. Como solo hay **19 valores posibles de departamento (una constante, independiente de n)**, para cada registro alcanza con compararlo contra los códigos 1, 2, ..., 19 (por ejemplo mediante una cadena de comparaciones “¿depto(r) = 1? ¿= 2? ...” o, más eficientemente, una búsqueda binaria entre los 19 valores posibles, en $O(\log 19) = O(1)$ comparaciones) para determinar a cuál de las 19 listas enlazadas debe unirse, y luego enlazarlo al final de esa lista en $O(1)$ (con un puntero de cola por lista).

```

Bucket-Sort-Comparaciones(registros[1..n]):
  Cabeza[1..19], Cola[1..19] = punteros NULOS (19 listas encadenadas vacías)
  para cada registro r en registros:                                // 0(n) iteraciones
    d = depto(r)
    encontrar el balde b tal que d = valor_referencia[b]           // ≤19 comparaciones,
O(1)                                                                // 0(1)
    enlazar r al final de la lista b (usando Cola[b])              // 0(1), solo punteros
    salida = concatenar las 19 listas en orden                      // 0(19) enlaces de
  punteros = 0(1)
  devolver salida

```

Como hay a lo sumo 19 valores posibles (constante, no crece con n), determinar a qué balde pertenece cada registro toma **$O(1)$ comparaciones por registro** (no $O(\log n)$: 19 es una constante fija, independiente de n), y el resto (enlazar en la lista, concatenar las 19 listas) es aritmética de punteros, $O(1)$ por operación. Tiempo total: **$O(n)$** , y el criterio de ordenamiento se determinó exclusivamente por comparaciones entre códigos de departamento (nunca se usó el código como dirección de memoria). ■

(C) RELACIÓN CON LA COTA INFERIOR $\Omega(N \log N)$: QUÉ HIPÓTESIS FALLA

La demostración de la cota $\Omega(n \log n)$ del Ejercicio 2 depende crucialmente de que el algoritmo debe ser capaz de producir, en el peor caso, **cualquiera de las $n!$ permutaciones posibles** de n elementos **todos distintos entre sí** — es decir, que el “espacio de resultados posibles” que el algoritmo debe distinguir tiene tamaño $n!$.

Acá esa hipótesis **no se cumple**: los códigos de departamento toman valores en un conjunto de tamaño **19, constante e independiente de n** — no son n valores arbitrarios y distintos, sino que (para $n > 19$, por el principio del palomar) necesariamente hay **repeticiones masivas** de código. El algoritmo de bucket sort no necesita distinguir entre dos registros del mismo departa-

mento (los deja en el orden en que llegaron, sin comparar entre sí quién “va primero” salvo por estabilidad trivial de la inserción); el número de resultados distintos que el algoritmo necesita ser capaz de producir no es $n!$, sino, a lo sumo, la cantidad de formas de **asignar cada uno de los n registros a uno de 19 baldes** — que es 19^n , no $n!$.

Y aquí está el punto preciso donde falla la demostración de la $\Omega(n \log n)$: el argumento de (b)-(c) del Ejercicio 2 necesita **$\log_2(\text{cantidad de resultados posibles}) = \log_2(n!) = \Theta(n \log n)$** comparaciones. Con solo 19^n resultados posibles en vez de $n!$:

$$\log_2(19^n) = n \cdot \log_2(19) = \Theta(n) \text{ — lineal, no } \Theta(n \log n).$$

Esto es exactamente coherente con que el algoritmo del ítem (b) logra $O(n)$ comparaciones: cada registro requiere solo $O(1) = O(\log_2 19)$ comparaciones (una constante independiente de n) para determinar su balde, y hay n registros, dando $O(n)$ total — muy por debajo de la cota $\Omega(n \log n)$, sin contradicción, porque esa cota **solo aplica a algoritmos que deben poder distinguir entre las $n!$ ordenaciones de n claves arbitrarias y distintas**, hipótesis que el problema de los 19 departamentos no satisface. ■

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica
S7-E1	D&V con $q=2$ sobre elemento mayoritario; Lema de “mayoritario en el todo $\Rightarrow$ mayoritario en alguna mitad”; recurrencia (5.1)/(5.2) del libro
S7-E2	Adaptación de Merge-and-Count (puntero monótono) a una condición de comparación distinta ($A_i > 2B_j$); descomposición exacta $R(L) = R(A) + r + R(B)$
S8-E1	D&V con $q=1$ y combinación $O(1)$ (no $O(n)$); invariante de “rango local” preservado por eliminación simétrica de mitades; recurrencia $Q(k) = Q(k/2) + O(1) \rightarrow \Theta(\log n)$ (extensión del caso $q=1$ del libro a costo constante)
S8-E2	Árbol de decisión para algoritmos de comparación; cota $2^{C(n)} \geq$ (resultados posibles); Fórmula de Stirling para $\log_2(n!) = \Theta(n \log n)$
S8-E3	Bucket/counting sort $O(n)$ no comparativo; versión comparativa con dominio de claves de tamaño constante (no n); por qué la cota $\Omega(n \log n)$ no aplica cuando el espacio de resultados posibles es 19^n y no $n!$

Para el examen: el patrón más transferible de este práctico es **plantear $T(n)$ correctamente identificando q y n/b antes que nada, y demostrar corrección con una hipótesis inductiva explícita** (casi siempre expresada como invariante sobre los parámetros de la llamada recursiva, como en S8-E1) — exactamente lo que sugiere el enunciado del Ejercicio 1 de la semana 8. La cota inferior $\Omega(n \log n)$ (S8-E2/E3) es también un patrón recurrente: cualquier algoritmo que “parece” violar una cota inferior conocida vale la pena revisarlo contra las **hipótesis exactas** de la demostración de esa cota (acá: claves distintas, dominio no acotado) — es un tipo de pregunta típica de examen (“¿por qué esto no contradice tal cota?”).